

Task5: Microservices with Consul

Виконала: Іванченко Марія

GitHub: <https://github.com/mariiaivanchenko/Software-Architecture.git>

Завдання базується на функціоналі розробленому у попередньому завданні, і є його подальшим розвитком. До системи необхідно додати *Consul*, який буде виконувати роль *Service Register*, *Service Discovery* та *Config Server*.

Запуск елементів

Спочатку скачала усе необхідного Consul.

```
brew install consul  
docker pull consul:1.14.0
```

Наступним кроком модифікувала попередній *docker-compose.yml*, тепер він створює також consul, який графічно можна переглянути на <http://localhost:8500>.

Внесені зміни у *docker-compose.yml*, додала *consul*:

```
services:  
  consul:  
    image: consul:1.14.0  
    container_name: consul  
    ports:  
      - "8500:8500"  
      - "8600:8600/udp"  
    command: agent -dev -client=0.0.0.0
```

Внесені зміни також у файл запуску:

```
echo "Running utils..."  
python3 utils.py &
```

Вимоги:

1. Всі мікросервіси мають самостійно динамічно реєструватись при старті у *Consul*, кожного з сервісів може бути запущено кілька екземплярів:

- ***facade-service***
- ***logging-service***
- ***messages-service***

Для цього перед запуском кожного сервісу він одразу реєструє себе за допомогою функції з файлу *utils.py*.

```
register_service("facade-service", 1, FACADE_PORT)  
atexit.register(lambda: deregister_service("facade-service", 1, FACADE_PORT))  
  
def register_service(service_name, member_id, port):  
    """  
    Function to register services to consul.  
    """  
    consul = Consul()  
    consul.agent.service.register(  
        name=f"{service_name}",  
        service_id=f"{service_name}-{member_id}-{port}",  
        address="127.0.0.1",  
        port=port,  
    )
```

2. При звертанні ***facade-service*** до ***logging-service*** та ***messages-service***, IP-адреси (і порти) мають зчитуватись ***facade-service*** з *Consul*. Немає бути

задано в коді чи конфігураціях статичних значень адрес.

При запиті GET на **facade-service** він обирає доступні сервіси за назвою серед "здорових" за допомогою *Consul*.

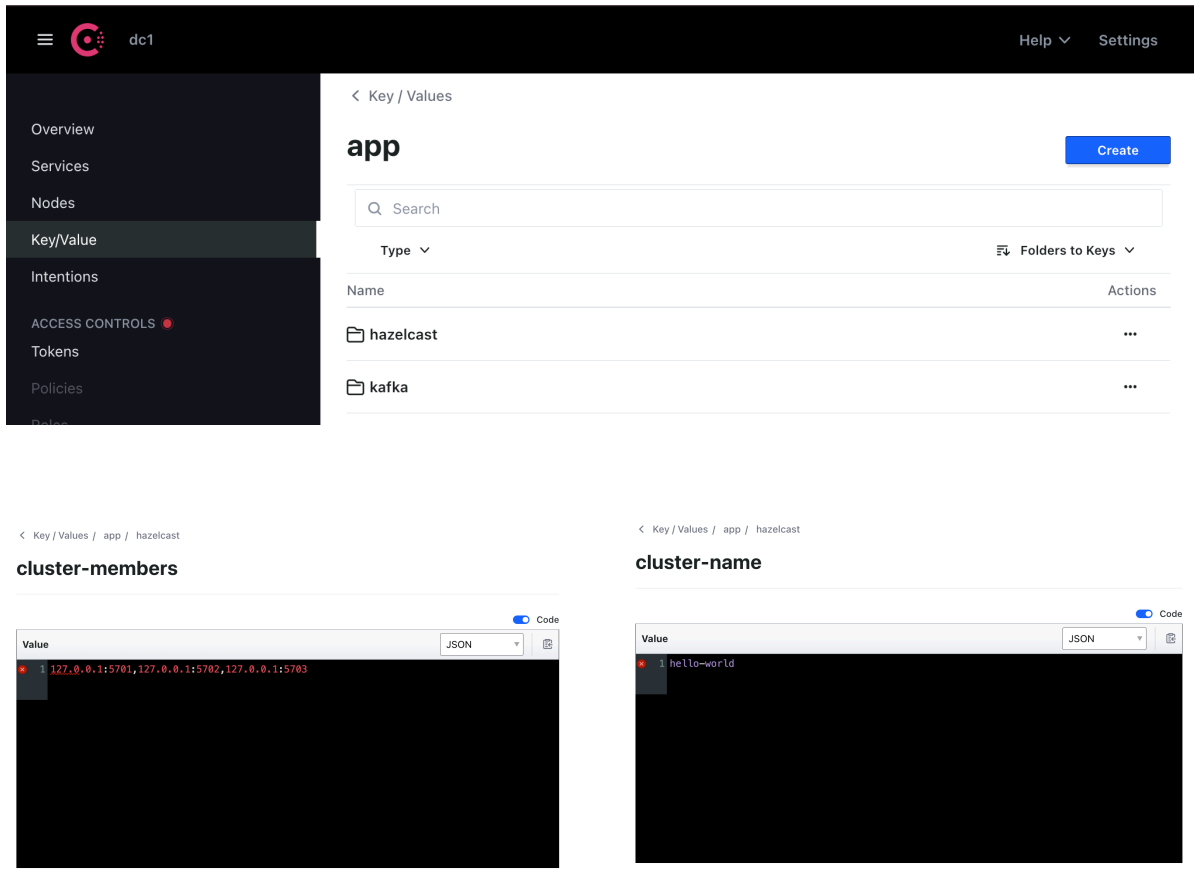
```
logging_url = choose_service("logging-service")
messages_url = choose_service("messages-service")

def choose_service(service_name):
    """
    Function to choose a service dynamically from Consul.
    """
    consul = Consul()
    health_services = consul.health.service(service_name)
    health_services = [
        "http://"
        + str(elem["Service"]["Address"])
        + ":"
        + str(elem["Service"]["Port"])
        + "/"
        + elem["Service"]["Service"]
        for elem in health_services[1]
    ]
    print("health_services: ", health_services)
    return random.choice(health_services)
```

3. Налаштування для клієнтів Hazelcast мають зберігатись як key/value у *Consul* і зчитуватись **logging-service**.

```
CLUSTER_NAME = read_element_kv("app/hazelcast/cluster-name")
CLUSTER_MEMBERS = read_element_kv("app/hazelcast/cluster-members").

save_element_kv("app/hazelcast/cluster-name", "hello-world")
save_element_kv("app/hazelcast/cluster-members", "127.0.0.1:5701,127.0.0.1:5702")
```



4. Налаштування для Message Queue (адреса, назва черги, ...) мають зберігатись як key/value у *Consul* і зчитуватись ***facade-service*** та ***messages-service***.

```

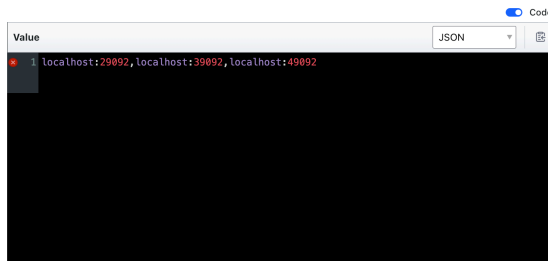
TOPIC_NAME = read_element_kv("app/kafka/topic-name")
BOOTSTRAP_SERVERS = read_element_kv("app/kafka/bootstrap-servers").s

save_element_kv("app/kafka/topic-name", "messages-topic")
save_element_kv("app/kafka/bootstrap-servers", "localhost:29092,localhost:29092")

```

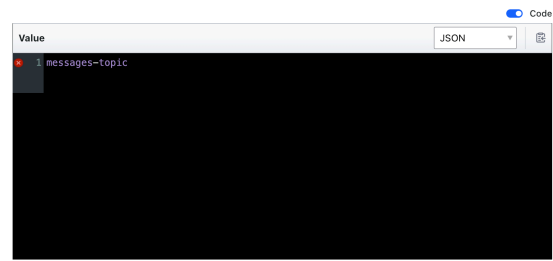
< Key / Values / app / kafka

bootstrap-servers



< Key / Values / app / kafka

topic-name

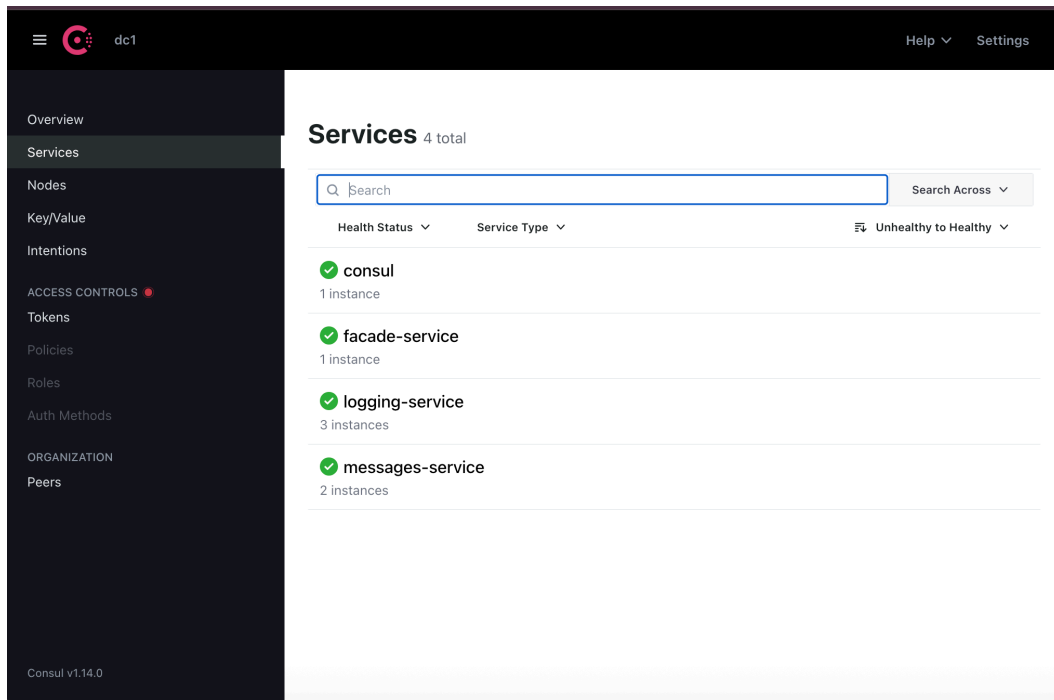


- Продемонструвати, що у випадку відключення екземпляру певного мікросервісу, це буде відображатись у Consul (відключений екземпляр сервісу змінить статус), а виклики будуть перенаправлятись до інших працюючих екземплярів.

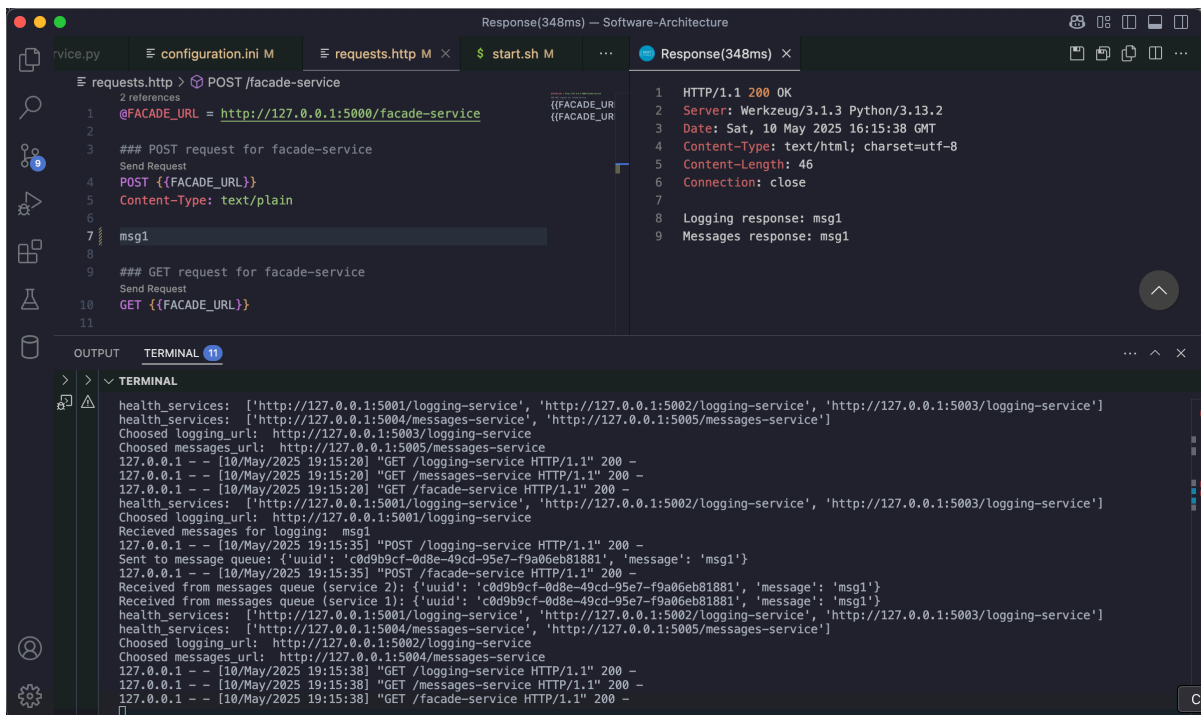
Спершу запускаємо та перевіряємо чи всі добре зареєструвались:

```
./start.sh
```

```
6 [URLS]
7 consul_url = http://localhost:8500
8
OUTPUT TERMINAL 11
> > TERMINAL
* Serving Flask app 'logging-service'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5002
Press CTRL+C to quit
2025-05-10 19:13:23.921 [ INFO] [hz.heuristic_einstein.priority-generic-operation.thread-0] [c.h.c.i.p.t.AuthenticationMessageTask]: [127.0.0.1]:5703 [hello-world] [5.5.0] Received auth from Connection[id=6, /127.0.0.1:5703->/127.0.0.1:62089, qualifier=null, endpoint=[127.0.0.1]:62089, remoteUid=90872f62-6ff8-46ed-ba7f-ca39263ef9bf, alive=true, connectionType=PYH, planeIndex=-1], successfully authenticated, clientUid: 90872f62-6ff8-46ed-ba7f-ca39263ef9bf, client name: hz.client_0, client version: 5.5.0
2025-05-10 19:13:23.984 [ INFO] [hz.jovial_hermann.priority-generic-operation.thread-0] [c.h.c.i.p.t.AuthenticationMessageTask]: [127.0.0.1]:5701 [hello-world] [5.5.0] Received auth from Connection[id=5, /127.0.0.1:5701->/127.0.0.1:62090, qualifier=null, endpoint=[127.0.0.1]:62090, remoteUid=90872f62-6ff8-46ed-ba7f-ca39263ef9bf, alive=true, connectionType=PYH, planeIndex=-1], successfully authenticated, clientUid: 90872f62-6ff8-46ed-ba7f-ca39263ef9bf, client name: hz.client_0, client version: 5.5.0
2025-05-10 19:13:23.997 [ INFO] [hz.pensive_mccarthy.priority-generic-operation.thread-0] [c.h.c.i.p.t.AuthenticationMessageTask]: [127.0.0.1]:5702 [hello-world] [5.5.0] Received auth from Connection[id=6, /127.0.0.1:5702->/127.0.0.1:62091, qualifier=null, endpoint=[127.0.0.1]:62091, remoteUid=90872f62-6ff8-46ed-ba7f-ca39263ef9bf, alive=true, connectionType=PYH, planeIndex=-1], successfully authenticated, clientUid: 90872f62-6ff8-46ed-ba7f-ca39263ef9bf, client name: hz.client_0, client version: 5.5.0
* Serving Flask app 'logging-service'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5003
Press CTRL+C to quit
Starting messages services...
* Serving Flask app 'messages-service'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5004
Press CTRL+C to quit
* Serving Flask app 'messages-service'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5005
Press CTRL+C to quit
```



Наступний крок перевірити чи працюють запити GET та POST:



Видалила:

- `flask --app logging-service run --host 127.0.0.1 --port=5002`

```
ps aux | grep flask
kill 62038
```

```

.venv+ Software-Architecture git:(micro_consul) x ps aux | grep flask
maria 61941 1.0 0.1 33732876 9272 s024 S+ 1:35AM 0:02.40 /usr/local/Cellar/python@3.13/3.13.2/Frameworks/Python.framework/Versions/3.13/Resources/Pyt
hon.app/Contents/MacOS/Python /Users/maria/Desktop/Task5/Software-Architecture/.venv/bin/flask --app logging-service run --host 127.0.0.1 --port=5001
maria 62038 0.9 0.1 33733900 9280 s024 S+ 1:35AM 0:02.18 /usr/local/Cellar/python@3.13/3.13.2/Frameworks/Python.framework/Versions/3.13/Resources/Pyt
hon.app/Contents/MacOS/Python /Users/maria/Desktop/Task5/Software-Architecture/.venv/bin/flask --app logging-service run --host 127.0.0.1 --port=5002
maria 62085 0.9 0.1 33732876 9296 s024 S+ 1:35AM 0:02.33 /usr/local/Cellar/python@3.13/3.13.2/Frameworks/Python.framework/Versions/3.13/Resources/Pyt
hon.app/Contents/MacOS/Python /Users/maria/Desktop/Task5/Software-Architecture/.venv/bin/flask --app logging-service run --host 127.0.0.1 --port=5003
maria 62088 0.6 0.1 33746064 11208 s024 S+ 1:35AM 0:01.66 /usr/local/Cellar/python@3.13/3.13.2/Frameworks/Python.framework/Versions/3.13/Resources/Pyt
hon.app/Contents/MacOS/Python /Users/maria/Desktop/Task5/Software-Architecture/.venv/bin/flask --app messages-service run --host 127.0.0.1 --port=5004
maria 62094 0.3 0.1 33749136 11276 s024 S+ 1:35AM 0:01.68 /usr/local/Cellar/python@3.13/3.13.2/Frameworks/Python.framework/Versions/3.13/Resources/Pyt
hon.app/Contents/MacOS/Python /Users/maria/Desktop/Task5/Software-Architecture/.venv/bin/flask --app messages-service run --host 127.0.0.1 --port=5005
maria 62857 0.0 0.0 33597656 524 s025 R+ 1:36AM 0:00.01 grep --color=auto --exclude-dir=.svn --exclude-dir=.idea --exclude-dir=.tox --exclude-dir=.venv --exclude-dir=.hg
maria 61939 0.0 0.1 33730676 7880 s024 S+ 1:35AM 0:01.10 /usr/local/Cellar/python@3.13/3.13.2/Frameworks/Python.framework/Versions/3.13/Resources/Pyt
hon.app/Contents/MacOS/Python /Users/maria/Desktop/Task5/Software-Architecture/.venv/bin/flask --app facade-service run --host 127.0.0.1 --port=5000
.venv+ Software-Architecture git:(micro_consul) x kill 62038

```

Відповідний екземпляр зник:

The screenshot shows the Consul web interface. On the left is a sidebar with navigation options: Overview, Services, Nodes, Key/Value, Intentions, ACCESS CONTROLS, Tokens, Policies, Roles, Auth Methods, ORGANIZATION, and Peers. The main content area is titled 'logging-service' and shows a list of service instances. There are two instances listed:

- logging-service-0-5001**: No service checks, All node checks passing, ID: b9013be982fb, Address: 127.0.0.1:5001.
- logging-service-2-5003**: No service checks, All node checks passing, ID: b9013be982fb, Address: 127.0.0.1:5003.